DATA VISUALIZATION

Data visualization is the graphical representation of data to help understand patterns, trends, and relationships. Python offers powerful libraries like Pandas and Seaborn to simplify data visualization and analysis.

SEABORN

Seaborn is a visualization library in Python built on top of Matplotlib. It is more advanced and provides default styles, attractive color palettes, and better-looking graphs. Seaborn offers different categories of plots such as relational plots, categorical plots, distribution plots, regression plots, and matrix plots.

Let us see the different plots in each category.

Seaborn Plot Categories

- **Relational Plots:** scatterplot(), lineplot(), relplot()
- **Categorical Plots:** barplot(), countplot(), boxplot(), violinplot(), swarmplot(), pointplot(), catplot()
- **Distribution Plots:** histplot(), kdeplot(), rugplot(), distplot()
- **Regression Plots:** regplot(), lmplot()
- **Matrix Plots:** heatmap(), clustermap()

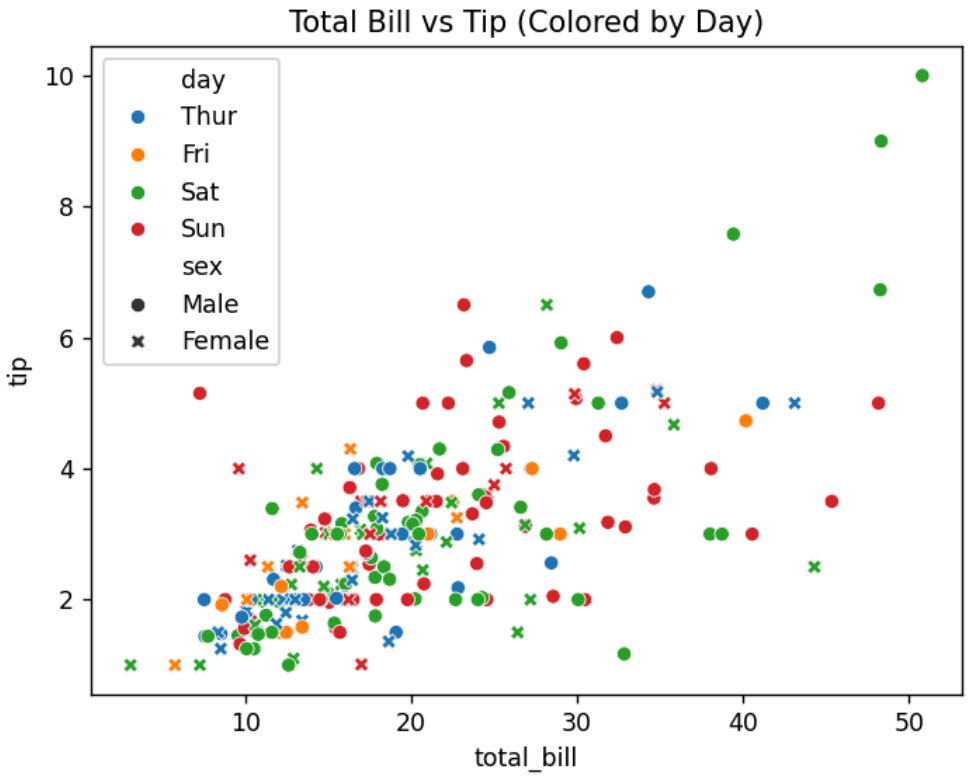
SAMPLE GRAPHS

SCATTER PLOT

Code Snippet:

```
import seaborn as sns
import matplotlib.pyplot as plt
tips = sns.load_dataset("tips")
sns.scatterplot(x="total_bill", y="tip", hue="day", style="sex", data=tips)
plt.title("Total Bill vs Tip (Colored by Day)")
plt.show()
```

Output:



Description:

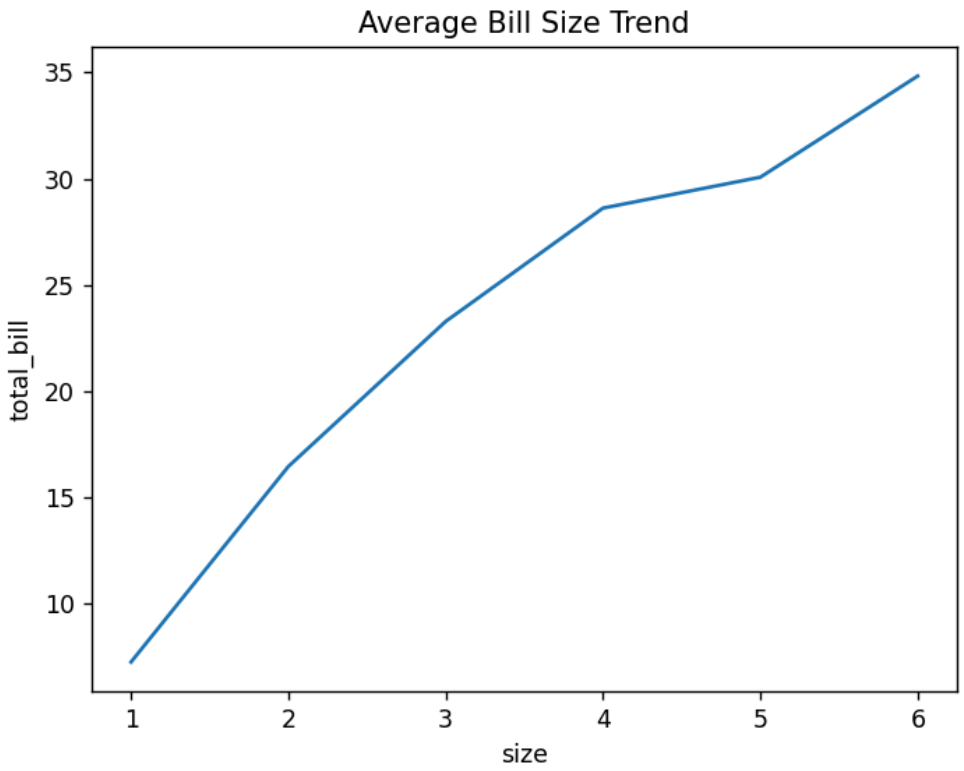
This scatter plot shows the relationship between total bill and tip amount. The hue and style parameters add extra dimensions to visualize categories clearly.

LINE PLOT

Code Snippet:

```
import seaborn as sns
import matplotlib.pyplot as plt
tips = sns.load_dataset("tips")
sns.lineplot(x="size", y="total_bill", data=tips, estimator='mean', ci=None)
plt.title("Average Bill Size Trend")
plt.show()
```

Output:



Description:

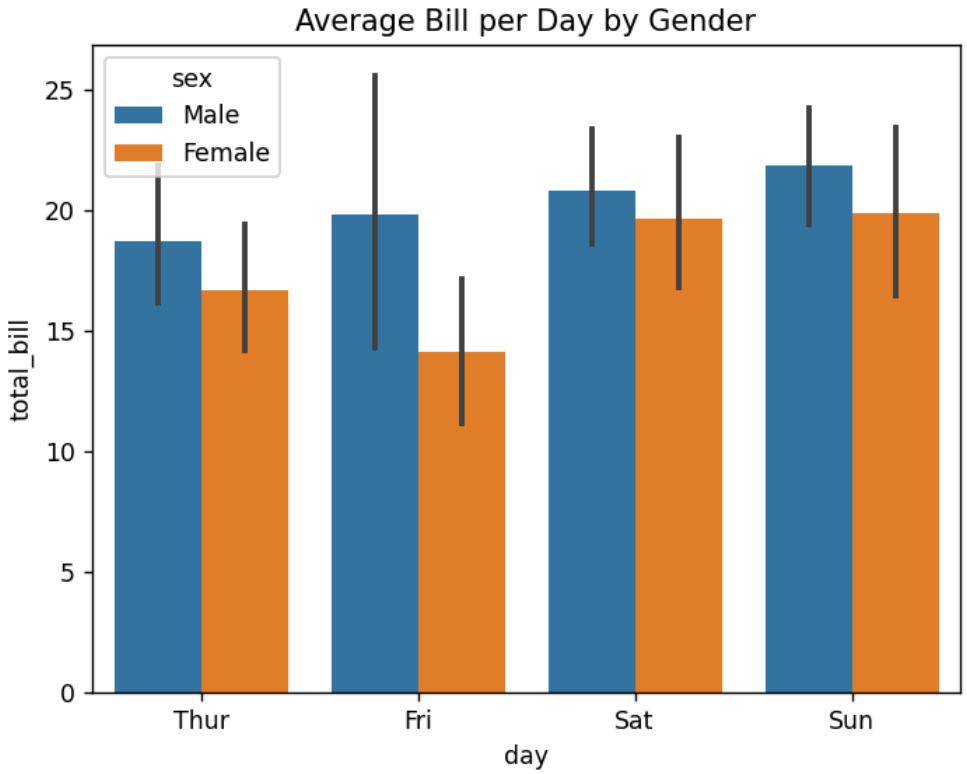
This line plot shows the average total bill based on group size. It helps in understanding trends using aggregated data instead of raw values.

BAR PLOT

Code Snippet:

```
import seaborn as sns
import matplotlib.pyplot as plt
tips = sns.load_dataset("tips")
sns.barplot(x="day", y="total_bill", hue="sex", data=tips)
plt.title("Average Bill per Day by Gender")
plt.show()
```

Output:



Description:

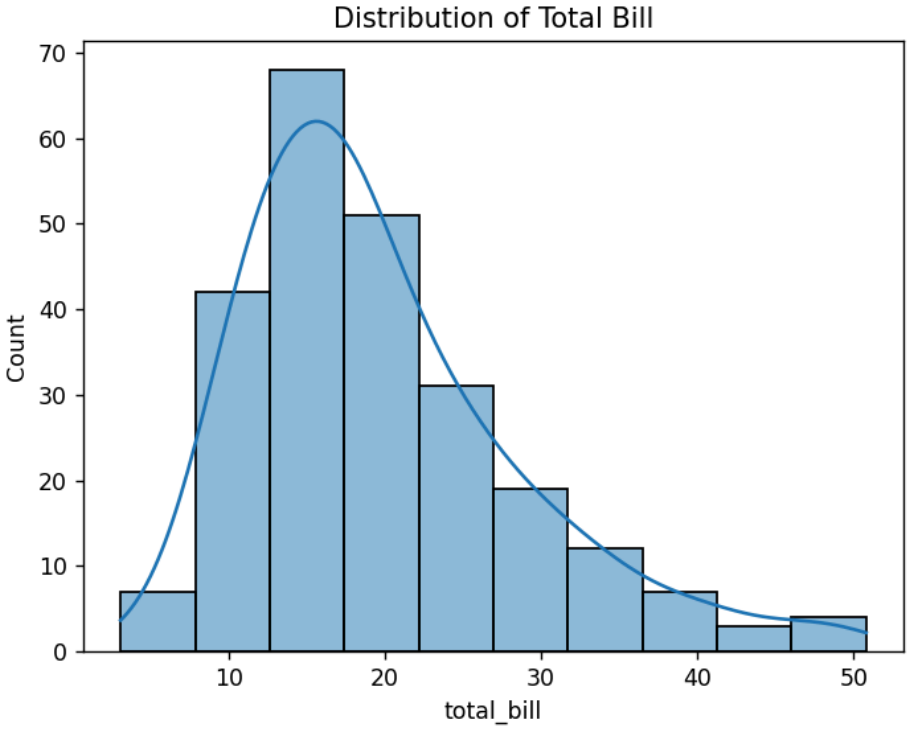
This bar plot compares average total bill across days and gender. It automatically calculates statistical estimates and displays confidence intervals.

HISTOGRAM

Code Snippet:

```
import seaborn as sns
import matplotlib.pyplot as plt
tips = sns.load_dataset("tips")
sns.histplot(tips["total_bill"], kde=True, bins=10)
plt.title("Distribution of Total Bill")
plt.show()
```

Output:



Description:

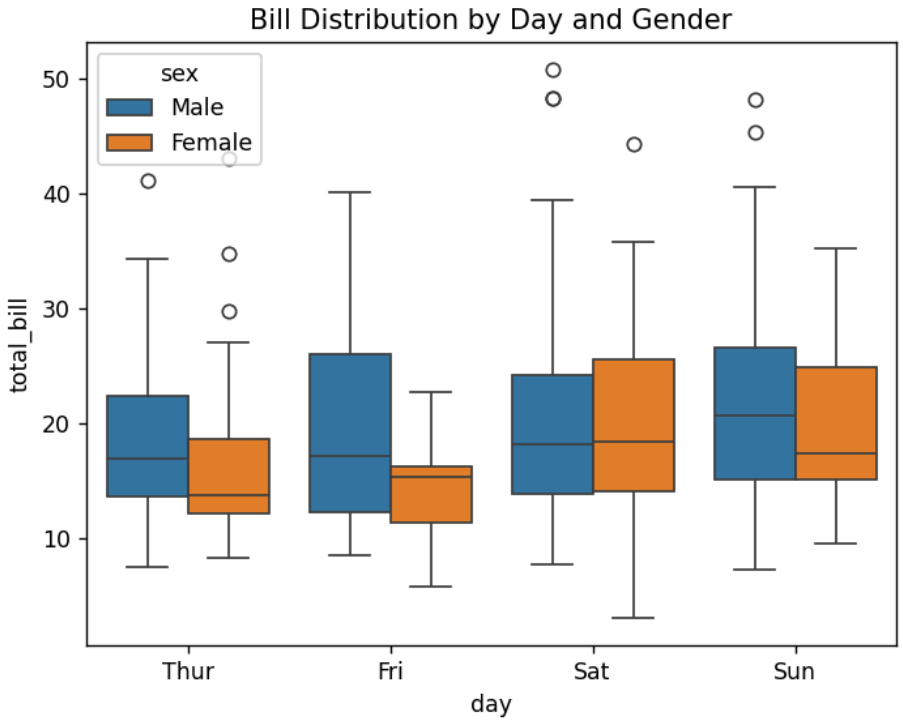
This histogram shows the distribution of total bill values along with a KDE curve. The KDE helps visualize the probability density smoothly.

BOX PLOT

Code Snippet:

```
import seaborn as sns
import matplotlib.pyplot as plt
tips = sns.load_dataset("tips")
sns.boxplot(x="day", y="total_bill", hue="sex", data=tips)
plt.title("Bill Distribution by Day and Gender")
plt.show()
```

Output:



Description:

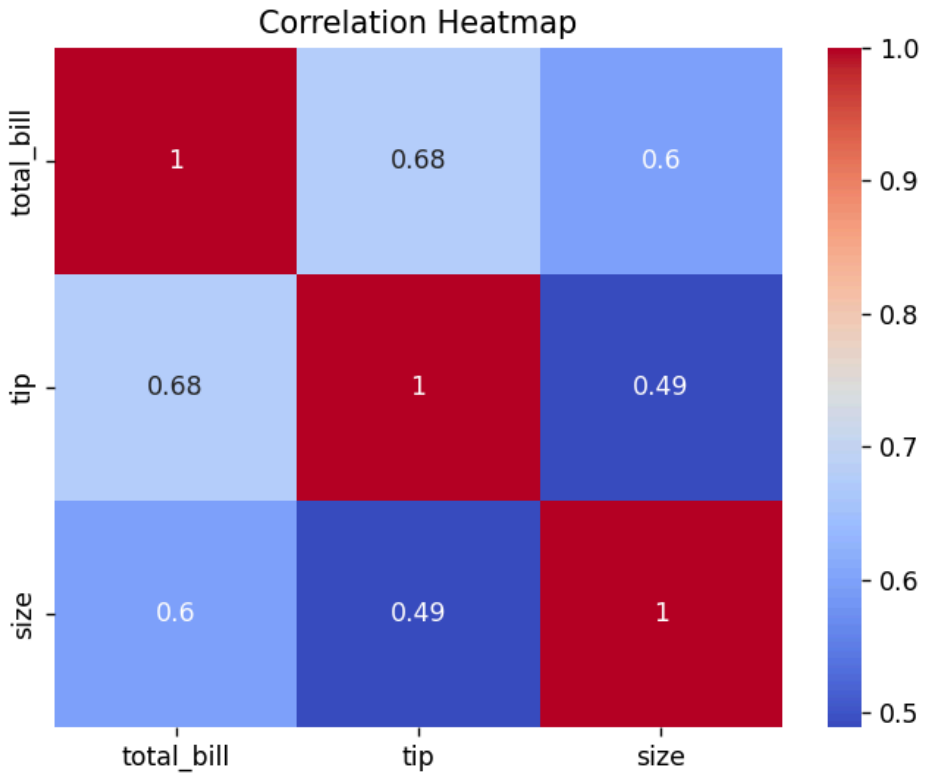
This box plot shows how total bill values vary across days and gender. It highlights median, quartiles, and outliers effectively.

HEATMAP

Code Snippet:

```
import seaborn as sns
import matplotlib.pyplot as plt
tips =sns.load_dataset("tips")
corr = tips.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```

Output:



Description:

This heatmap visualizes correlations between numerical variables. Color intensity represents the strength of relationships between features.

PANDAS

Pandas is a powerful Python library used for data manipulation and analysis. It also provides built-in visualization capabilities that are simple and effective for quick data exploration. Pandas uses Matplotlib internally for plotting graphs.

Pandas supports different types of plots such as line plot, bar plot, scatter plot, histogram, pie chart, and more.

Let us see the different plots.

TYPES OF PLOTS IN PANDAS:

- line plot
- bar plot
- scatter plot
- histogram

- pie chart
- box plot

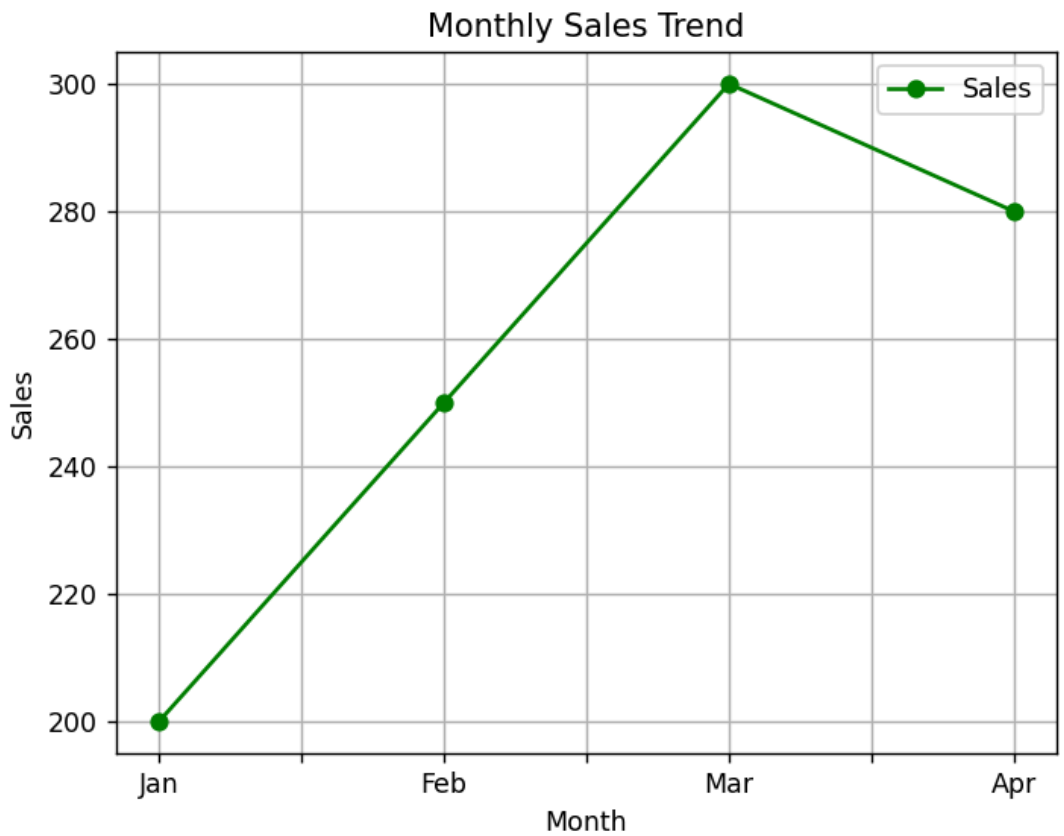
SAMPLE GRAPHS

LINE PLOT

Code Snippet:

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    'Month': ['Jan', 'Feb', 'Mar', 'Apr'],
    'Sales': [200, 250, 300, 280]
}
df = pd.DataFrame(data)
df.plot(x='Month', y='Sales', kind='line', marker='o', color='green')
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.grid(True)
plt.show()
```

Output:



Description:

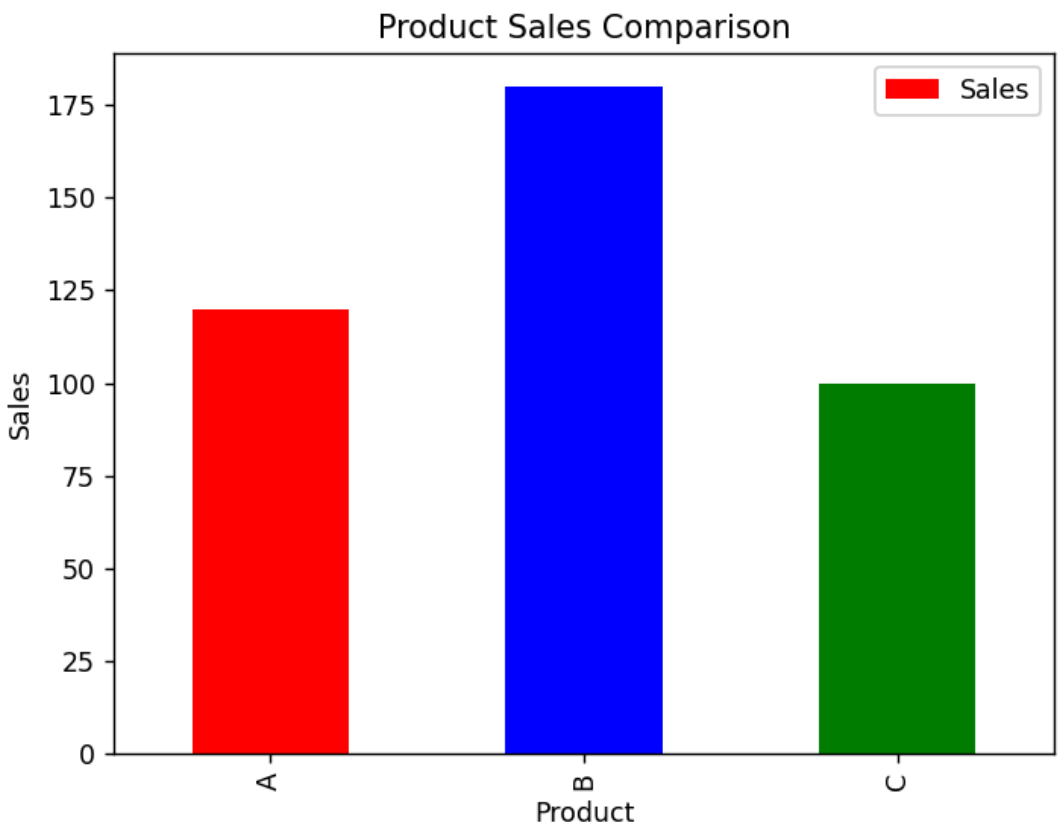
This line plot shows the trend of sales over different months. Markers and grid lines improve readability and make it easier to track changes over time.

BAR PLOT

Code Snippet:

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    'Product': ['A', 'B', 'C'],
    'Sales': [120, 180, 100]
}
df = pd.DataFrame(data)
df.plot(x='Product', y='Sales', kind='bar', color=['red', 'blue', 'green'])
plt.title("Product Sales Comparison")
plt.xlabel("Product")
plt.ylabel("Sales")
plt.show()
```

Output:



Description:

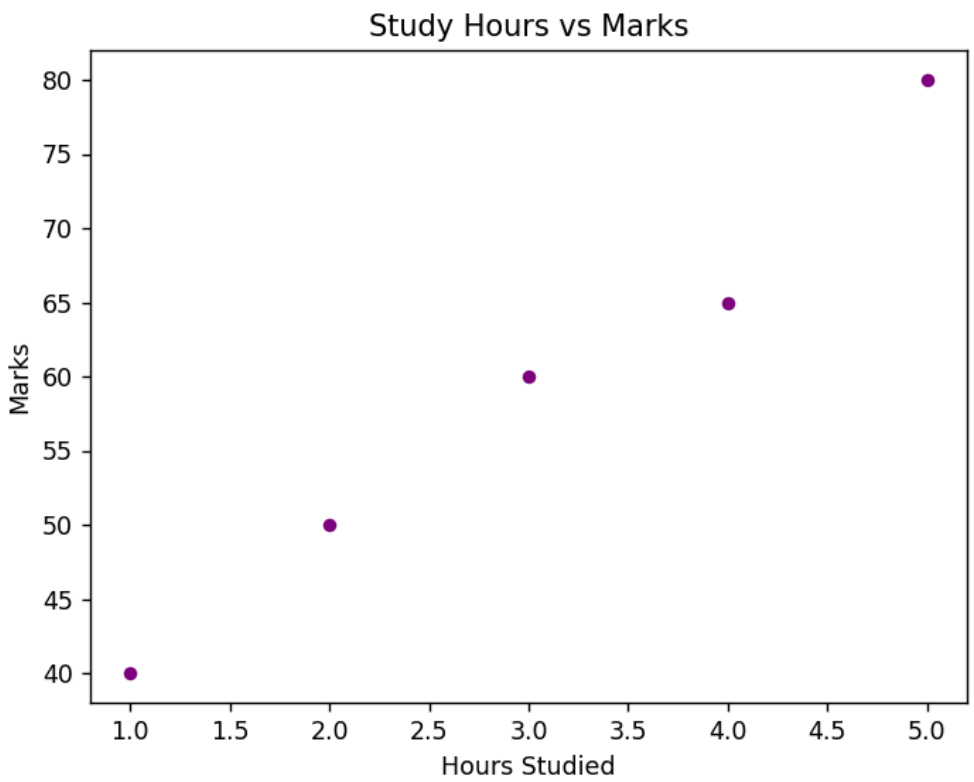
This bar chart compares sales across different products. Custom colors make the chart more visually appealing and help distinguish categories easily.

SCATTER PLOT

Code Snippet:

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    'Hours': [1, 2, 3, 4, 5],
    'Marks': [40, 50, 60, 65, 80]
}
df = pd.DataFrame(data)
df.plot(kind='scatter', x='Hours', y='Marks', color='purple')
plt.title("Study Hours vs Marks")
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.show()
```

Output:



Description:

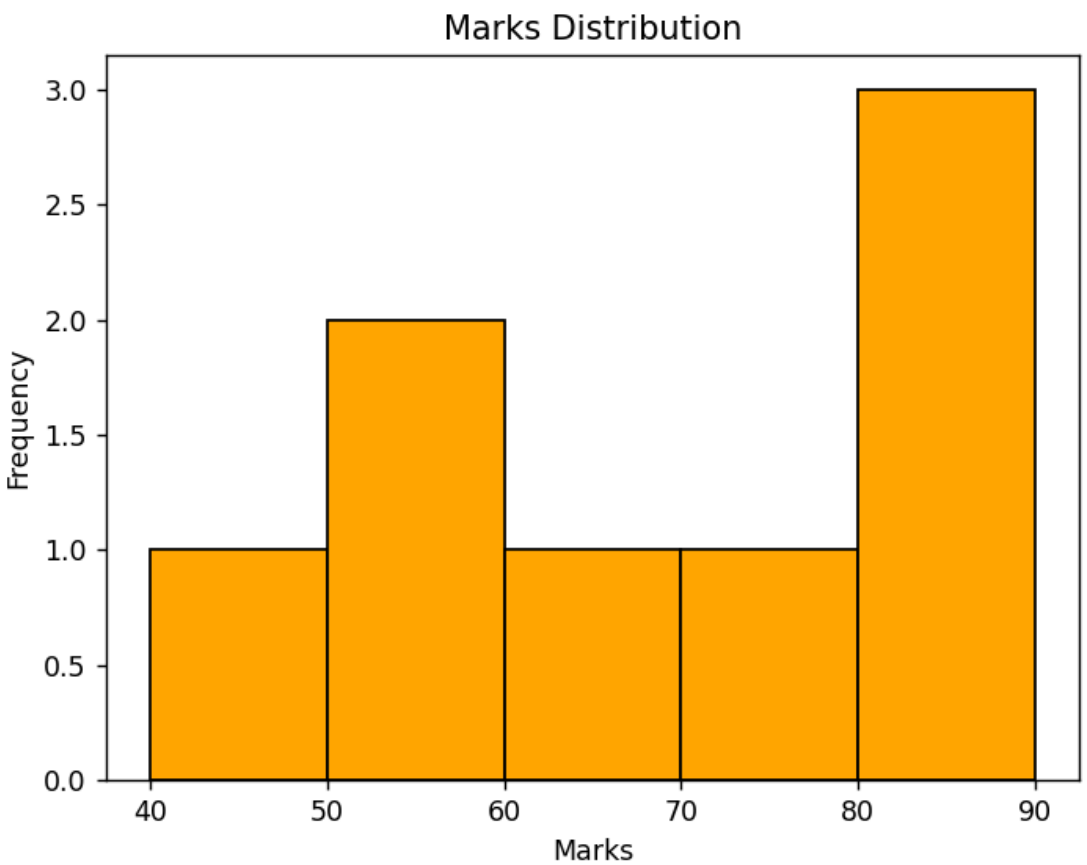
This scatter plot shows the relationship between study hours and marks. It helps identify whether increasing study time leads to better performance

HISTOGRAM

Code Snippet:

```
import pandas as pd
import matplotlib.pyplot as plt
data = {'Marks': [40, 50, 50, 60, 70, 80, 80, 90]}
df = pd.DataFrame(data)
df['Marks'].plot(kind='hist', bins=5, color='orange', edgecolor='black')
plt.title("Marks Distribution")
plt.xlabel("Marks")
plt.show()
```

Output:



Description:

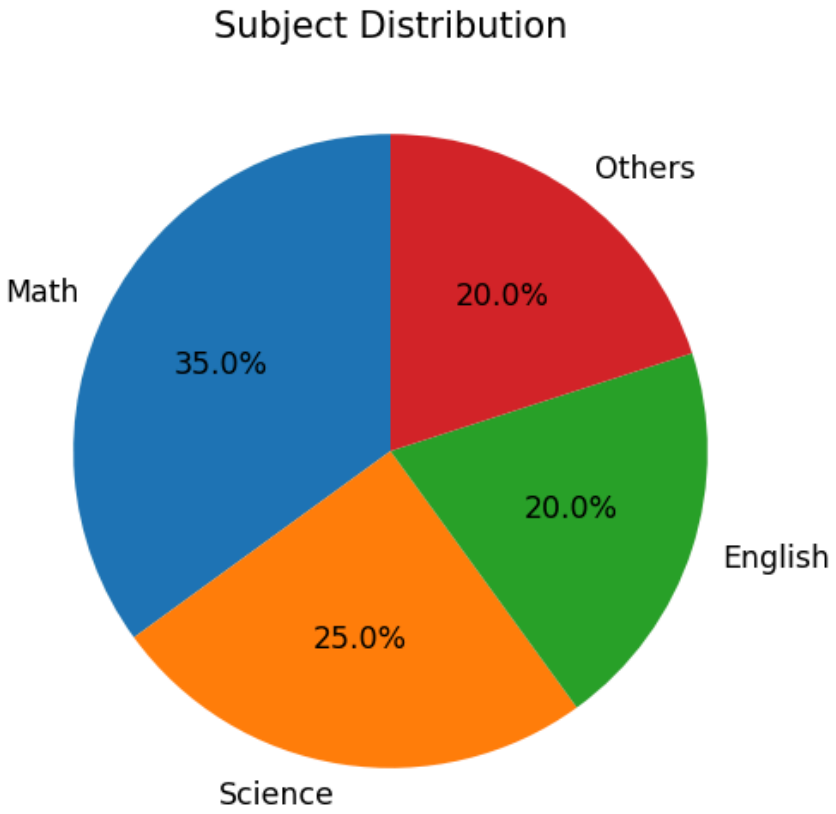
This histogram displays how marks are distributed among students. Bins divide the data into ranges, helping visualize frequency patterns.

PIECHART

Code Snippet:

```
import pandas as pd
import matplotlib.pyplot as plt
data = pd.Series([35, 25, 20, 20], index=['Math', 'Science', 'English', 'Others'])
data.plot(kind='pie', autopct='%1.1f%%', startangle=90)
plt.title("Subject Distribution")
plt.ylabel("")
plt.show()
```

Output:



Description:

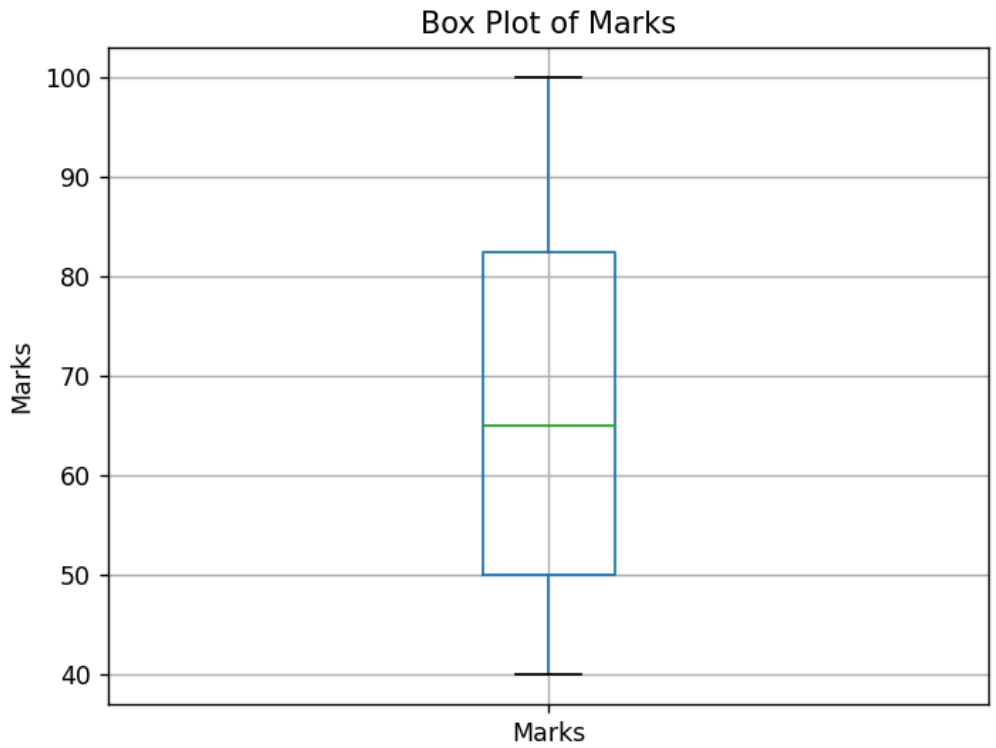
This pie chart represents the proportion of marks across subjects. Percentage labels make it easy to understand the contribution of each category.

BOX PLOT

Code Snippet:

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.DataFrame({
    'Marks': [40, 50, 50, 60, 70, 80, 90, 100]
})
df.boxplot()
plt.title("Box Plot of Marks")
plt.ylabel("Marks")
plt.show()
```

Output:



Description:

Box plots in Pandas display the distribution of data using quartiles such as minimum, maximum, median, and interquartile range. They also help identify outliers, which appear as points outside the whiskers.

COMPARISON OF PANDAS AND SEABORN

Pandas:

Core Library:
Pandas is mainly used for data analysis and provides basic visualization using Matplotlib.

Ease of Use:
Very simple and beginner-friendly for quick plotting.

Customization:
Limited customization; depends on Matplotlib for advanced changes.

Integration:
Works well with NumPy and Matplotlib for data analysis tasks.

Advantages of Pandas:

- Easy and quick to use
- Direct plotting from datasets
- Best for simple visualizations
- Useful for data analysis

Seaborn:

High-Level Interface:
Seaborn is built on Matplotlib and provides advanced visualization features.

Statistical Plotting:
Supports statistical graphs like box plot, heatmap, and regression plots.

Aesthetics:
Provides attractive default styles and color palettes.

Integration with Pandas:
Works directly with Pandas DataFrames.

Advantages of Seaborn:

- Easy to create advanced plots
- Attractive and clean visuals

- Less code required
- Best for data analysis and insights

Overall Comparison:

Pandas is mainly used for data manipulation and quick visualizations, making it ideal for beginners and fast analysis. Seaborn, on the other hand, is more suitable for advanced and statistical visualizations with better design and aesthetics.

Pandas is preferred when working with data and needing simple plots quickly, whereas Seaborn is preferred when deeper insights and visually appealing graphs are required. In many cases, both libraries are used together to achieve efficient data analysis and visualization.

Resources:

- Pandas: https://pandas.pydata.org
- Seaborn: https://seaborn.pydata.org